一位淘宝工程同学的大模型LoRA微调尝试

原创 用户场景营销技术 大淘宝技术 2025年9月24日 01:35 浙江 13人

本文主要介绍了大模型时代下，如何通过 LoRA（Low-Rank Adaptation）这一参数高效微调技术，实现对大模型的轻量级定制。文章从微调的基本概念出发，详细阐述了 LoRA 的原理、优势与局限性，并结合本地原生实现（Transformers + PEFT）和百炼平台两种方式，展示了在小样本、低资源场景下的实战流程。结果表明，LoRA 能以极低的计算成本让通用大模型有效学习业务知识，显著提升其在特定任务中的表现，真正实现“让大模型懂业务”，推动 AI 从“可用”走向“好用”。

01 前言

如今大模型时代，深刻重塑了很多行业的业务形态与技术架构。从智能客服到内容生成，从代码辅助到推荐系统，大模型正以前所未有的速度渗透到互联网技术的各个领域，成为驱动创新的核心引擎。其强大的泛化能力和上下文理解水平，使得许多过去需要复杂工程与规则设计的任务，如今只需一次“提示”即可完成。

然而，尽管大模型展现出惊人的通用能力，对大多数开发者和业务方而言，它仍是一个“黑盒”——我们只能调用其预训练时所掌握的知识与行为模式，难以干预其内部逻辑，也无法直接引导模型适应特定场景的表达习惯、术语体系或业务规则。在生产环境中，这种“能力固定”的特性带来了明显的局限：模型输出可能偏离业务预期，难以保证一致性，更无法随业务演进而持续进化。

面对这一挑战，模型微调（Fine-tuning）提供了一条关键路径。尤其是以 LoRA 等为代表的参数高效微调技术（PEFT），在不改变主干模型的前提下，仅通过训练少量新增参数，即可实现对大模型行为的精准定制。这不仅大幅降低了计算成本与技术门槛，更让业务开发者能够基于自身积累的高质量数据，赋予通用大模型“专属”的业务理解能力。本文将围绕 LoRA微调技术展开，探讨其在小样本、低资源场景下的实践价值，展示如何利用业务数据对大模型进行轻量级定制，真正实现“让大模型懂业务”，推动 AI 从“可用”走向“好用”，从“通用”迈向“专属”。

02 微调

什么是微调

大模型的微调（Fine-tuning），是指在一个已经预训练好的大模型（比如 GPT、 DeepSeek、BERT、Llama等）基础上，针对特定任务、领域或数据进行进一步训练。微调通常通过在目标数据集上继续训练模型权重，使模型更适应该任务，从而提升下游任务的表现。

简单来说，预训练让模型掌握通用知识，微调让模型适应某个具体用例。LLM 微调是一个有监督学习过程，即有监督微调(SFT)，通常语言模型的初始训练是无监督的，但微调是有监督的。

微调的分类

按照目标任务

- 标准微调（任务微调）：针对某一具体任务进行微调（如分类、问答、摘要等）。
- 领域微调：针对某一特定领域数据进行微调（如医疗、法律、金融等）。
- 指令微调：通过大量的「指令-响应」数据集（如Alpaca, Baize等）对模型进行微调，使其能够按照自然语言指令完成多种任务。

按参数调整方式

- 全参数微调（Full-parameterTuning）：对所有模型参数进行更新。
- 部分参数微调（Repurposing）：只更新部分参数，比如最后一层、分类头等。
- 参数高效微调（ParameterEfficientFine-Tuning, PEFT）：引入少量额外可训练参数，冻结原模型绝大部分参数，如LoRA、Adapter、Prefix Tuning等，主要目的是用很少量新参数即实现任务适配。

常见微调方法

全参数微调

- 直接在目标数据集上继续训练所有模型参数，效果好但耗资源大，对硬件要求高，不易迁移。

参数高效微调（PEFT）

- LoRA（Low-Rank Adaptation）：通过在部分参数上引入低秩矩阵，仅训练这些新增的低秩参数，大大减少训练所需参数量和显存消耗。
- Adapter：在原有网络层之间插入小型新网络模块并仅训练这些模块参数。
- PrefixTuning / PromptTuning：在输入或模型内部插入可训练的参数向量（前缀、提示），主网络参数保持不变。



什么是 LoRA

LoRA(Low-RankAdaptationof LLMs)，即 LLMs的低秩适应，是参数高效微调最常用的方法。

原理

LoRA的核心思想是：冻结预训练的基座模型（Base Model），在其某些层（如注意力中的 Q、K、V投影）上，额外添加低秩矩阵（A和 B）作为可训练参数，用于学习任务特定的增量调整。

- 原始基座模型的权重 全程保持冻结、未修改。
- 训练过程中，只有 LoRA 引入的少量新增参数（如 lora_A, lora_B）被更新。

LoRA的本质就是用更少的训练参数来近似 LLM全参数微调所得的增量参数，从而达到使用更少显存占用的高效微调。

LoRA的核心思路就是，直接 freeze基座模型的全部参数，然后额外给模型增加一个扰动的 module，来模拟 finetune 之后参数改变的效果。如此一来，只要这些扰动的 module参数量够小且推理够快，就不会影响模型整体的 inference 成本，且可以大幅地缩减模型 finetune的开销，因为我们只需要训练极小量的一部分参数即可。

基座模型只参与训练过程中的前向传播，反向传播更新参数的只有 LoRA的插入的adapter参数。

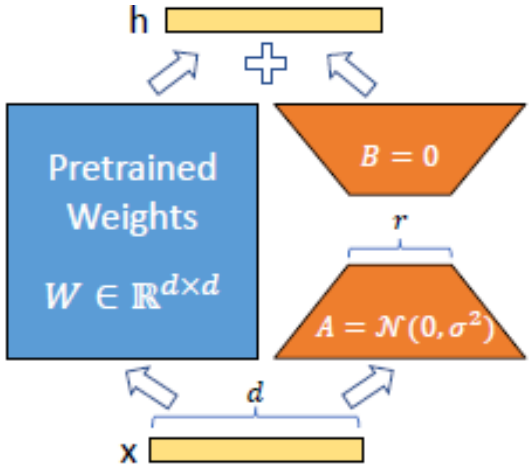


Figure 1: Our reparametrization. We only train A and B.

- 保留原来预训练好的权重 W不动；
- 旁路插入一个低秩矩阵 BA，对 W 的输出做一个可学习的微小增量；
- 只训练 A、B 这两组很小的参数，就能让模型适配新任务。

上面是简化的示意图，实际的 Transformer由很多线性投影（如 Q、K、V、O、FFN 的W_1、W_2 等）组成，每个都是自己的 $d_1 \times d_2$ 矩阵。 LoRA 是把这些大矩阵 W 逐个冻结，然后在它们上面叠加一个低秩增量 $\Delta W = BA$。 因此，严格来说是对模型里的每个大矩阵都做一次 rank = r（远小于 d_1, d_2）的低秩更新。

训练省资源的根本原因

- 原来若要微调 W，需要存和反向传播 $d_1 \cdot d_2$ 个参数；
- LoRA只训练 A($r \times d_2$)和 B($d_1 \times r$)，参数量变成 $2 \cdot r \cdot \max(d_1, d_2)$，当 $r \ll \min(d_1, d_2)$ 时可省 100× 乃至 10 000× 的显存和梯度计算；
- W冻结 → 不必存动量/梯度，也不用进行前后两次 matmul比较差分，大幅降低显存与算力。

- 推理过程

核心机制：权重增量叠加（Weight Addition）。

LoRA（Low-RankAdaptation）在推理时，并不是“绕道”或“替换”，而是：将 LoRA学习到的增量权重，加到原始预训练模型的对应权重上。

数学表达：

对于一个原始权重矩阵 $W \in Rd \times kW \in Rd \times k$，LoRA引入低秩分解：

$\Delta W = AB$其中 $A \in Rd \times r$，$B \in Rr \times k$，$r \ll d\Delta W = AB$其中 $A \in Rd \times r$, $B \in Rr \times k$, $r \ll d$

推理时的实际权重为：

$W_{new}=W+\Delta W=W+AB$

然后使用 W_{new}进行前向计算。

两种推理模式：

模式	说明	特点
1. 推理时动态叠加（On-the-fly）	加载原始模型 + LoRA 权重，实时计算 $W+\Delta W$	可用，但慢
2. 合并后推理（MergedInference）	将 LoRA 权重 合并到原始模型中，生成一个新模型，直接推理	推理更快、更稳定

一般采用合并后生成新的模型来进行推理，同时可以保留原始模型。

- 优劣势

LoRA 的优势

优势	说明
资源成本低	仅需训练少量参数（如 QKV投影层），节省 GPU显存和算力
训练速度快	梯度计算集中在小模块，训练效率显著提升
存储友好	一个基础模型 + 多个 LoRA权重（每个几 MB~几百 MB），便于版本管理和部署
灵活切换任务	可动态加载不同 LoRA权重，实现多业务场景快速切换
保护原始模型	原模型冻结，避免灾难性遗忘，保证通用能力

LoRA 的局限性

劣势	说明

性能略低于全参数微调	在复杂任务上可能稍逊于全量微调，但差距通常可控（<5%）
依赖基础模型质量	LoRA无法修复基础模型的根本缺陷，微调效果受限于预训练能力
适配层选择敏感	效果受插入位置（如仅 attention、FFN等）影响，需实验调优
推理需额外集成	需支持 LoRA加载的推理框架（如 HuggingFace + PEFT、vLLM、llama.cpp等）

实战

LoRA微调目前有多种方式实现，Transformers + PEFT原生实现，LLaMA-Factory、Axolotl等框架实现，以及借助百炼等封装好的可视化界面平台实现。本文主要尝试了 Transformers + PEFT原生实现，以及公司生产中更有可能用到的百炼平台实现。

- 原生微调

使用 Transformers + PEFT原生实现。

环境搭建

1. transformers+peft+datasets

注意：几个库的版本之间有协同关系，随意安装版本可能导致不兼容问题，可在Hugging Face的官方文档中查看推荐版本。

代码：

```
pip install --upgrade torch==2.1.0+cpu torchvision==0.16.0+cpu --index-
pip install -U transformers==4.38.2 peft==0.10.0 datasets==2.18.0 \
            accelerate sentencepiece safetensors tqdm
```

2. 模型准备

这里考虑到微调是在 mac本地运行，算力有限，所以使用了较小的 DeepSeek-R1-Distill-Qwen-1.5B作为基座模型。使用 Hugging Face或 Modelscope等下载模型到本地即可。

数据处理

训练数据：

准备好数据文件 lora_seckill_qa.jsonl，格式使用如下 instruction-response格式。

```
{"instrucAon":"埋点时主要记录哪些事件？","response":"使用的是xx埋点框架，埋点有
{"instrucAon":"xxxxxxxxxxxxxxxxxx？","response":"xxxxxxxxxxxxxxx"}
```

处理脚本：

```
import json
from datasets import Dataset
from transformers import AutoTokenizer
# 正确读取jsonl
with open("lora_seckill_qa.jsonl", "r", encoding="utf-8") as f:
raw_data = [json.loads(line) for line in f if line.strip()]
# 如果是单轮格式可直接用
# dataset = Dataset.from_list(raw_data)
# 若是conversation 格式（如 [{"conversation":[...]}]），需展开
def conversation_to_list(item):
out = []
for turn in item["conversation"]:
instr = turn.get("system", "") + "\n" + turn["input"] if
turn.get("system") else turn["input"]
out.append({
"instruction": instr.strip(),
"response": turn["output"].strip()
})
return out
# 如果你的raw_data 已经是单轮格式就跳过这一段
all_samples = []
if "conversation" in raw_data[0]:
for d in raw_data:
all_samples.extend(conversation_to_list(d))
dataset = Dataset.from_list(all_samples)
else:
dataset = Dataset.from_list(raw_data)

# 保存分词器
tokenizer = AutoTokenizer.from_pretrained(
"/Users/shawn/Documents/AI-dev/models/deepseek/deepseek-ai/DeepSeek-R1
Distill-Qwen-7B",
trust_remote_code=True,
local_files_only=True
)
tokenizer.save_pretrained("./tokenizer")
# 保存处理后的数据集
dataset.save_to_disk("./processed_dataset_ms")
print("预处理完成，已保存！")
```

训练参数

训练脚本：

```
import torch
from datasets import load_from_disk
from transformers import AutoTokenizer, AutoModelForCausalLM,
TrainingArguments, Trainer, default_data_collator
from peft import get_peft_model, LoraConfig
import random
# 1. 加载数据集
dataset = load_from_disk("./processed_dataset_ms")
# 2. 样本随机打乱
dataset = dataset.shuffle(seed=42)
# 3. 分词器
tokenizer = AutoTokenizer.from_pretrained("./tokenizer", local_files_o
def generate_and_tokenize_prompt(batch):
texts = [
f"""<s>### Instruction:
{instruction}
### Response:
{response}
</s>"""
for instruction, response in zip(batch["instruction"],
batch["response"])
]
out = tokenizer(
texts,
max_length=256,
padding="max_length",
truncation=True,
add_special_tokens=False,
return_tensors=None,
)
# Loss 忽略paddingp.
out["labels"] = [
[tok if tok != tokenizer.pad_token_id else -100 for tok in label]
for label in out["input_ids"]
]
return out
tokenized_dataset = dataset.map(
generate_and_tokenize_prompt,
batched=True,
remove_columns=dataset.column_names,
desc="Tokenizing"
)
```

```
# 4. 加载基座模型（如设备有限可指定CPU/其他device）
model = AutoModelForCausalLM.from_pretrained(
"/Users/shawn/Documents/AI-dev/models/deepseek/deepseek-ai/DeepSeek-R1
Distill-Qwen-7B",
torch_dtype=torch.bfloat16, # MPS、A100 等建议用bfloat16
local_files_only=True,
trust_remote_code=True
)
# 5. LoRA 配置
lora_config = LoraConfig(
r=8,
lora_alpha=16,
target_modules=["q_proj", "v_proj", "k_proj", "o_proj"],
lora_dropout=0.05, # 或0, 稳定死记记忆；推荐小数据降dropout
bias="none",
task_type="CAUSAL_LM"
)
model = get_peft_model(model, lora_config)
model.print_trainable_parameters()
# 6. 训练参数
training_args = TrainingArguments(
output_dir="./results_ms",
per_device_train_batch_size=2,
gradient_accumulation_steps=1,
learning_rate=3e-4,
num_train_epochs=8,
logging_dir="./logs_ms",
save_steps=100,
save_total_limit=3,
logging_steps=10,
overwrite_output_dir=True,
report_to=None,
fp16=False
)
trainer = Trainer(
model=model,
args=training_args,
train_dataset=tokenized_dataset,
data_collator=default_data_collator,
)
# 7. 训练与自动采样输出
trainer.train()
# 8. 保存模型与tokenizer
model.save_pretrained("./deepseek-7b-lora_ms")
tokenizer.save_pretrained("./deepseek-7b-lora_ms")
print("训练完成。")
```

89

参数解析

LoRA配置参数（lora_config）

1. r=8

- 含义：LoRA中低秩矩阵的秩（rank），用于近似原始权重矩阵。
- 作用：控制新增参数量，显著降低训练显存消耗。
- 影响：
  - r太小 → 欠拟合，表达能力不足
  - r太大 → 接近全量微调，失去 LoRA优势
- 常见值：4~ 64，根据任务复杂度调整（如简单任务用 r=8，复杂任务可用 r=32）

2. lora_alpha=16

- 含义：LoRA权重的缩放因子，控制低秩矩阵对输出的影响强度。
- 作用：调节LoRA分支的贡献程度，类似“学习率系数”。
- 影响：
  - 推荐保持 alpha / r≥ 1（如 r=8, alpha=16→ 比值为 2）
  - 值过大 → 容易过拟合；值过小 → 影响微弱
- 建议：初试设置 alpha = 2 × r

3. target_modules=["q_proj","v_proj","k_proj","o_proj"]

- 含义：指定在哪些模块上应用 LoRA（通常是注意力层的投影层）。
- 作用：精准控制适配范围，节省计算资源。
- 常见选择：
  - LLaMA系列：["q_proj", "k_proj", "v_proj", "o_proj"]
  - BERT类：["query", "value", "key", "dense"]
- 影响：
  - 仅作用于部分模块 → 可能限制模型适应能力
  - 可根据模型结构扩展（如加入 up_proj, down_proj）

4. lora_dropout=0.05

- 含义：LoRA层的 Dropout概率，用于正则化。
- 作用：防止过拟合，提升泛化能力。
- 影响：
  - 值越大 → 正则化越强
  - 小数据集建议：0.1~ 0.2
  - 大数据集可设为 0或 0.05
- 注意：若训练不稳定，可尝试增加 dropout

5. bias="none"

- 含义：是否在 LoRA层中引入偏置项。
- 选项：
  - "none"：不加偏置（推荐起点）
  - "all"：所有模块加偏置
  - "lora_only"：仅 LoRA层加偏置
- 影响：
  - 加偏置可能略微提升效果，但增加参数量
- 建议：从 "none" 开始，效果不佳再尝试 "lora_only"

6. task_type="CAUSAL_LM"

- 含义：指定任务类型，影响损失函数和标签处理方式。
- 常用类型：
  - "CAUSAL_LM"：自回归语言模型（如 GPT、LLaMA生成任务）
  - "SEQ_CLS"：序列分类（如情感分析）
  - "TOKEN_CLS"：Token级分类（如命名实体识别）
  - "SEQ_2_SEQ_LM"：Seq2Seq任务（如翻译、摘要）
- 注意：必须与任务匹配，否则训练会出错

训练参数（training_args）

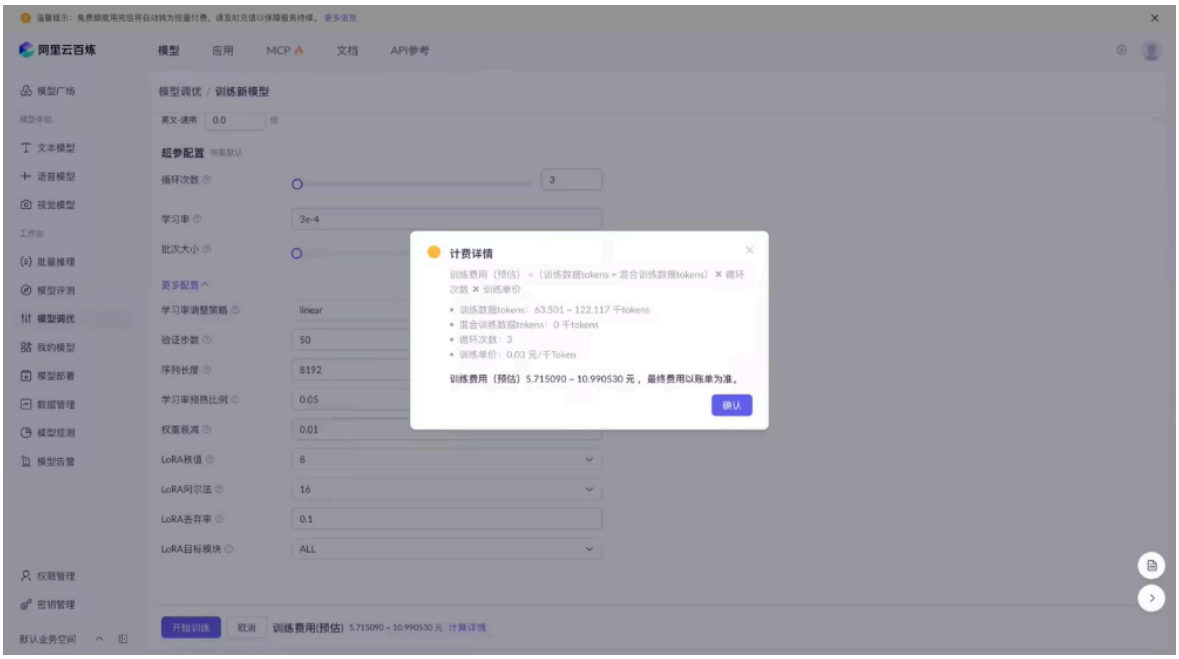
参数	值	说明
output_dir	"./results_ms"	保存模型检查点、日志等文件的目录
per_device_train_batch_size	2	每个设备上的 batchsize，显存不足时可减小
gradient_accumulation_steps	1	梯度累积步数，等效增大 batchsize（如设为 4→ 实际 batch = 8）
learning_rate	3e-4	学习率，LoRA推荐范围：1e-4~1e-3
num_train_epochs	8	训练轮数，一般从 3~10开始调整
logging_dir	"./logs_ms"	日志保存路径，可用于TensorBoard可视化

save_steps	100	每隔多少 step保存一次 checkpoint
save_total_limit	3	最多保留几个 checkpoint，自动删除旧的
logging_steps	10	每隔多少 step输出一次日志，便于监控
overwrite_output_dir	True	若目录存在则覆盖，避免手动清理
report_to	None	不连接远程报告工具（如 wandb、tensorboard）
fp16	False	是否启用 FP16半精度训练 ✅ 支持设备建议开启（节省显存 + 加速） ⚠️ 若开启需确保模型

场景	建议配置
小数据集微调	r=8，alpha=16，dropout=0.1，epochs=5~10
大数据集微调	r=16~32，alpha=32，dropout=0.05，lr=2e-4
显存受限	减小 batch_size，启用 fp16，使用 gradient_accumulation
快速实验	overwrite_output_dir=True ，logging_steps=10，save_steps=50

训练成本

使用1000条左右数据微调qwen3-14b的模型，一次训练成本预计 10元左右。可以看到模型微调本身的成本并不算很高，即使换参数量更大的模型以及更多的数据。后续的部署、推理成本才是大头。



测试脚本

```python
import torch
from transformers import AutoModelForCausalLM, AutoTokenizer
from peft import PeftModel
# 路径设置
BASE_PATH = "/Users/shawn/Documents/AI-dev/models/deepseek/deepseekai/
DeepSeek-R1-Distill-Qwen-7B"
LORA_PATH = "./deepseek-7b-lora_ms"
# 加载分词器
tokenizer = AutoTokenizer.from_pretrained(
BASE_PATH,
local_files_only=True,
trust_remote_code=True
)
# === 严格分离模型实例 ===
# 原始模型
model_base = AutoModelForCausalLM.from_pretrained(
BASE_PATH,
device_map="mps",
torch_dtype=torch.bfloat16,
local_files_only=True,
trust_remote_code=True
)
# 微调后的模型（Base + LoRA 适配器）
model_base_for_lora = AutoModelForCausalLM.from_pretrained(
BASE_PATH,
device_map="mps",
torch_dtype=torch.bfloat16,
local_files_only=True,
```

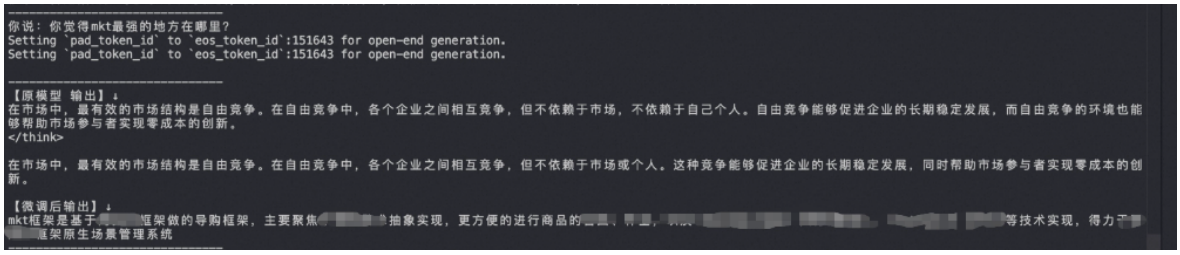
```
trust_remote_code=True
)
model_lora = PeftModel.from_pretrained(
model_base_for_lora,
LORA_PATH
)
def format_prompt_one_round(user_input: str) -> str:
return f"<s>### Instruction:\n{user_input}\n### Response:\n"
def generate_single(model, prompt, tokenizer, max_new_tokens=200):
inputs = tokenizer(prompt, return_tensors="pt")
for k, v in inputs.items():
inputs[k] = v.to(model.device)
with torch.no_grad():
outputs = model.generate(
**inputs,
max_new_tokens=max_new_tokens,
temperature=0.7,
top_p=0.9,
do_sample=True,
eos_token_id=tokenizer.eos_token_id
)
full_output = tokenizer.decode(outputs[0], skip_special_tokens=True)
reply = full_output[len(prompt):]
# 截断下一个分隔符，保证只输出新生成内容
for sep in ["<s>", "</s>", "###"]:
if sep in reply:
reply = reply.split(sep)[0]
return reply.strip()
def main():
print("="*40)
print("DeepSeek 模型微调前/后 单轮对话对比（严格模型物理分离版）")
print("="*40)
print("输入exit 退出。\n")
while True:
user_input = input("你说：").strip()
if user_input.lower() in {"exit", "quit"}:
print("对话结束~ 再见！")
break
prompt = format_prompt_one_round(user_input)
# 原始模型推理
base_reply = generate_single(model_base, prompt, tokenizer)
# LoRA 微调后模型推理
lora_reply = generate_single(model_lora, prompt, tokenizer)
print("\n------------------------------")
print("【原模型 输出】↓\n" + base_reply)
print("\n【微调后输出】↓\n" + lora_reply)
```

```
print("--------------------------------")
if __name__ == "__main__":
main()

```

测试结果 case：

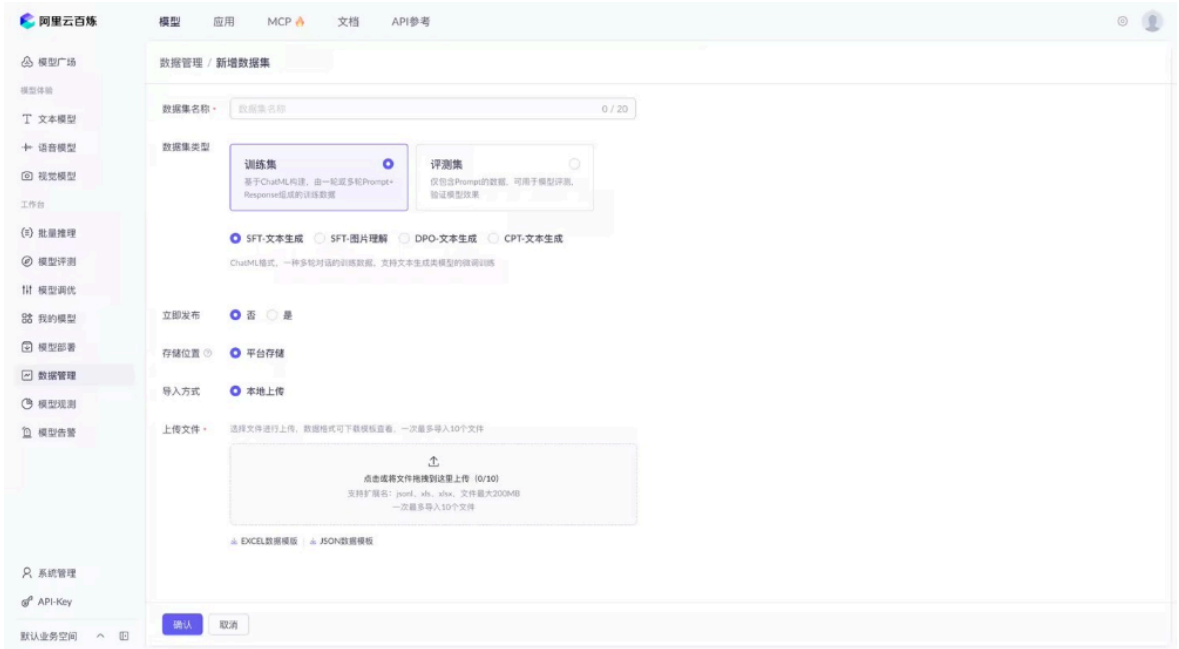
可以看到原模型完全无法感知到业务相关的知识，而微调后的模型已经有效的学到了（注：由于数据量小，且模型本身参数量也小，做演示的微调结果存在一定的过拟合）



- 百炼平台微调

数据准备

根据百炼的训练数据示例格式准备数据

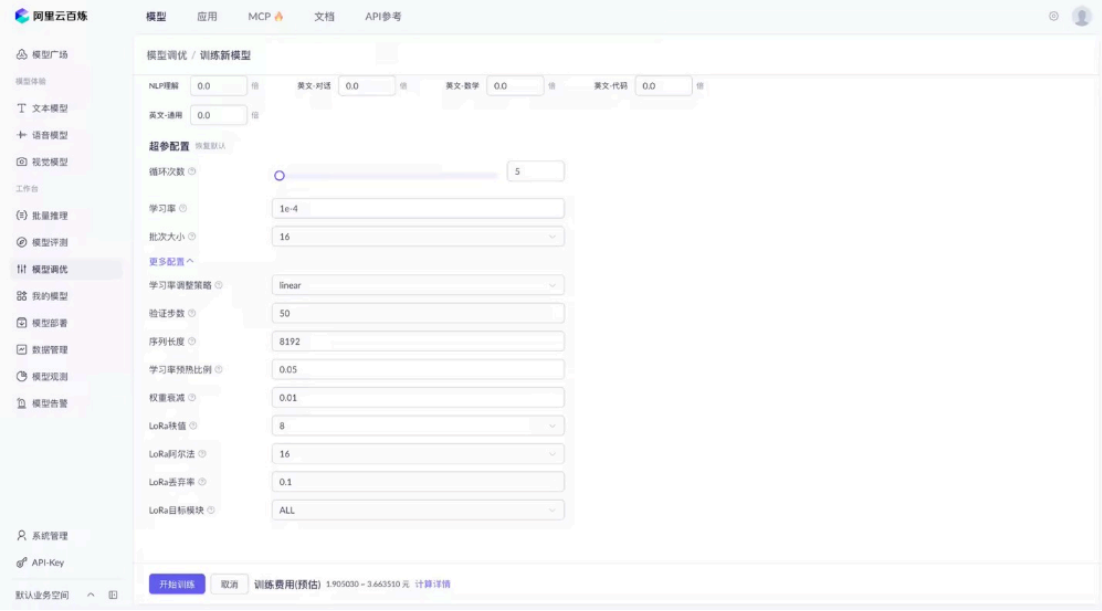


百炼训练数据格式

```
{“messages": [{"role": "system", "content": "You are a professional e-
title analyst. Given a long product title, output ONLY the core product
```

```
name (1-5 words) without any other text."}, {"role": "user", "content"
商品标题中提取最核心的商品主体，直接输出主体名，不要加其它词：【黑旗】心语肉松原味辣
味1kg 烘焙面包蛋糕寿司原料商用肉松小贝"}, {"role": "assistant", "content": "
"}]}
{"messages": [{"role": "system", "content": "You are a professional e-
title analyst. Given a long product title, output ONLY the core product
name (1-5 words) without any other text."}, {"role": "user", "content"
列商品标题中提取最核心的商品主体，直接输出主体名，不要加其它词：夏季竹枕片成人藤凉席冰
丝枕头套单人儿童竹枕席藤枕芯套一对拍2"}, {"role": "assistant", "content": "枕
```

训练参数

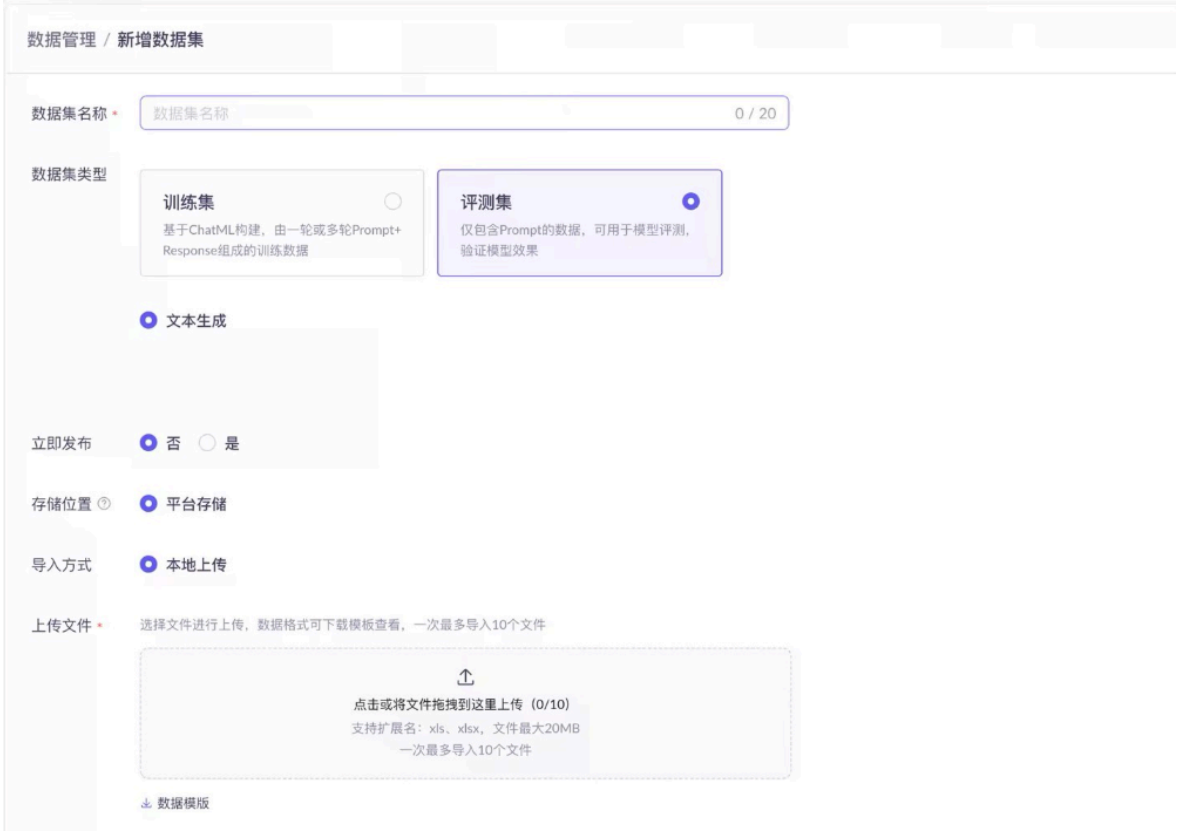


参数跟原生实现的参数基本一致，可以参考上述的训练参数解析及百炼的文档。

评测

1. 创建评测数据集

创建方式同训练数据，使用百炼模版

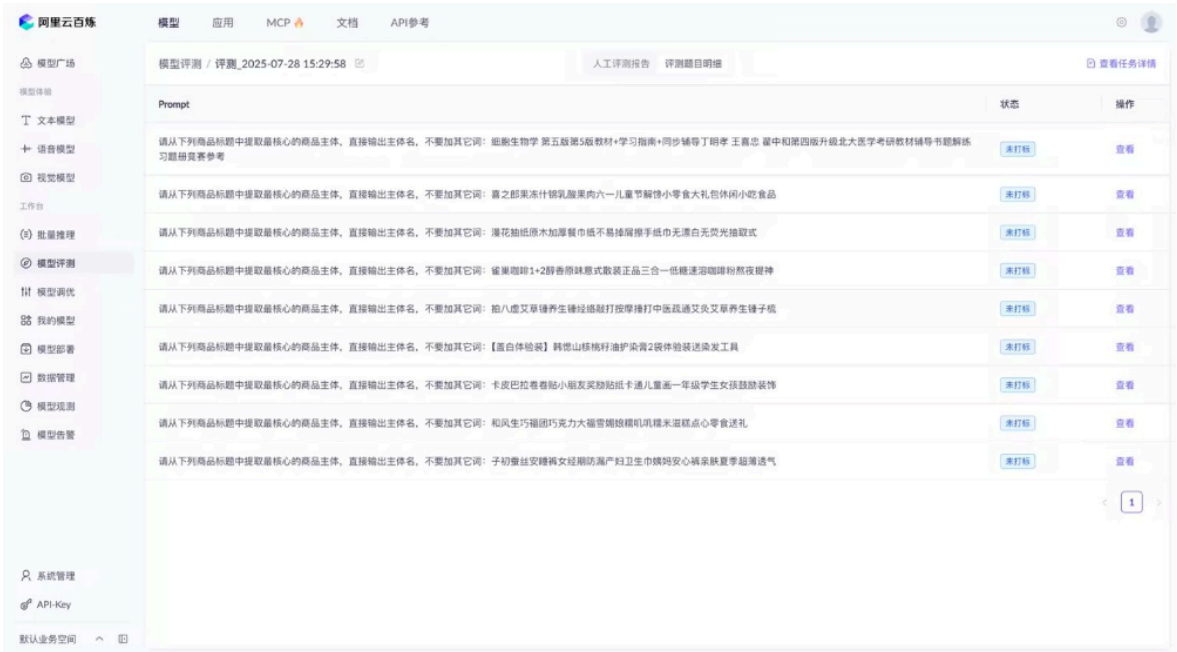


2.选择评测方式，开始评测

有三种评测方式

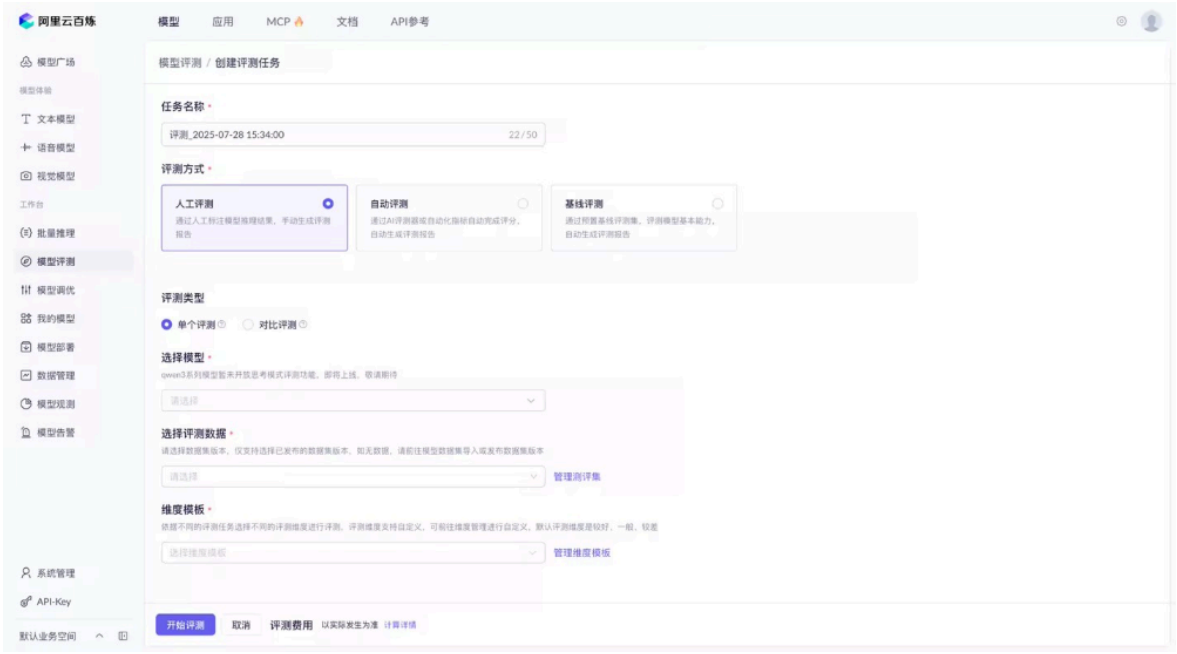
- 人工评测

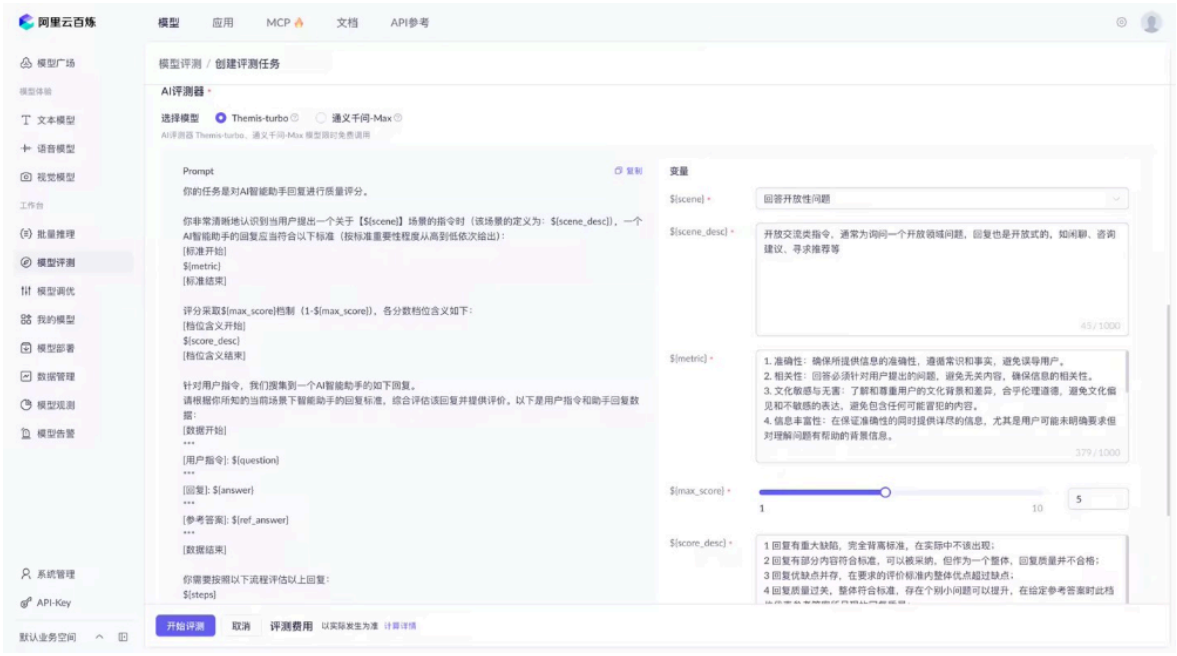
特点：经过人工校验打分，在很多应用场景下是相对准确的；同时比较主观，在一些场景下可能不够标准化，且人力消耗较高。

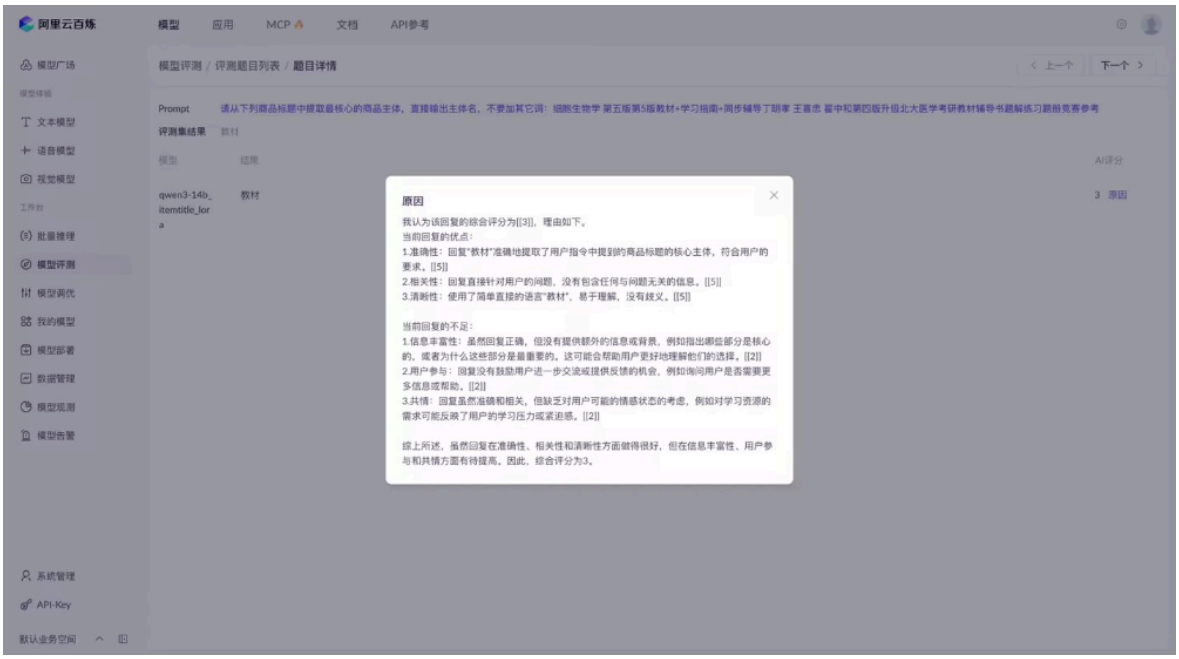


- 模型评测

特点：适合评测文本生成类，可以调整 prompt 来对自己微调后模型的回答要点进行打分，利用另一个大模型的能力，实现灵活打分，但是比较依赖模型的能力以及 prompt，同时分数有时也不够标准化。

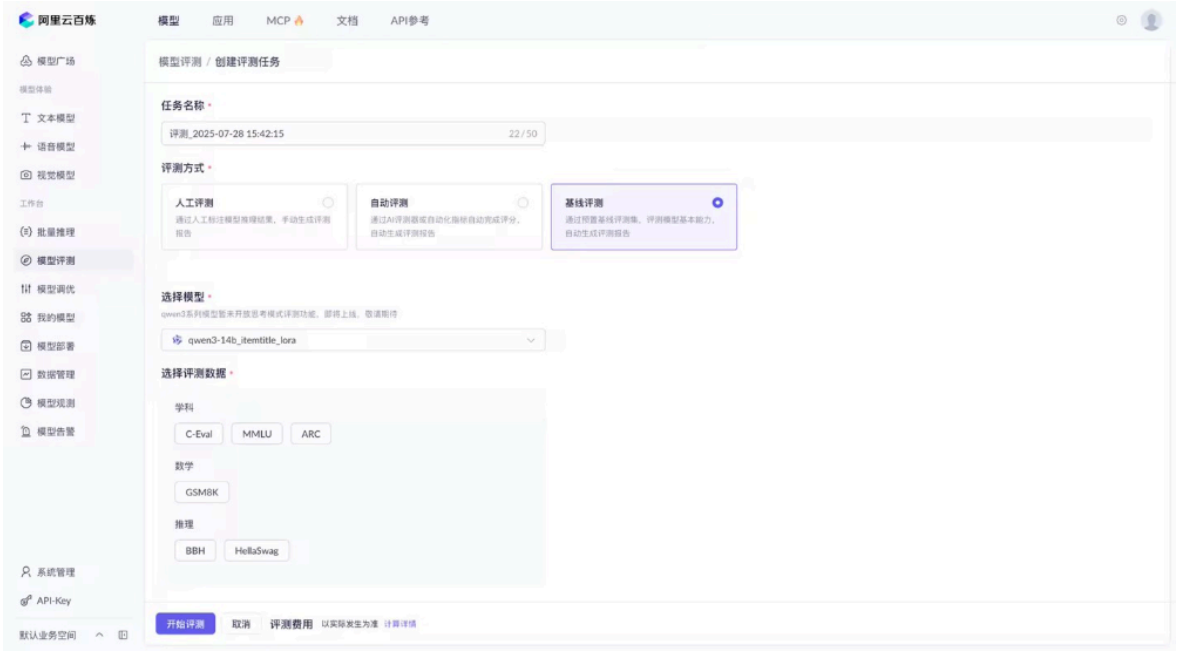






- 基线评测

通过预置的算法来对模型的特定方向能力进行标准化评测。



手动测试结果

左侧为微调后的模型，右侧为原始模型，其对商品主体识别的能力得到提升。



04 总结

LoRA（Low-RankAdaptation）微调技术为业务开发场景提供了高效、低成本的大模型定制化路径，展现出显著优势：

- 训练成本低，效果出色
  - 显存占用少：仅需微调少量新增参数，可在消费级 GPU上完成训练，降低硬件门槛。
  - 训练速度快：参数更新量小，支持快速迭代，适合业务团队敏捷试错。
  - 性能接近全量微调：在多种任务中表现优异，效果逼近全参数微调，性价比极高
- 部署灵活，易于管理
  - 适配器轻量化：LoRA 权重文件通常仅几十 MB，便于存储、传输与版本管理。
  - 多任务共享主干：支持“一个基础模型 + 多个 LoRA适配器”的模式，实现不同业务线共用底座模型，提升资源利用率。
- 稳定可靠，风险可控
  - 缓解过拟合：在小样本场景下表现更稳定，有效避免全参数微调带来的灾难性遗忘或过拟合问题。
  - 保护原始模型：不修改主干参数，仅通过低秩增量进行适配，提升模型鲁棒与可维护性。

核心价值：让大模型真正“懂业务”

LoRA 微调的轻量与高效，使得业务开发者能够基于自身积累的高质量业务数据（如客服对话、商品描述、内部流程文本等），对大模型进行快速定制化训练。这不仅大幅降低了 AI 落地的技术门槛，更让模型输出更可控、更精准、更贴近实际业务需求。未来，我们将持续探索 LoRA在真实业务场景中的应用，例如智能推荐、自动化文案生成、研发提效等方向，以数据驱动 + 模型微调的方式，助力业务增长与技术升级。

05 参考资料

- LoRA:Low-RankAdaptationofLargeLanguageModels：https://arxiv.org/abs/2106.09685
- QLoRA: Efficient Finetuning of Quantized LLMs：https://arxiv.org/abs/2305.14314
- P-Tuningv2:PromptTuningCanBeComparabletoFine-tuningUniversallyAcrossScalesandTasks：https://arxiv.org/abs/2110.07602
- 文献阅读：LoRA: Low-RankAdaptationofLargeLanguageModels：https://blog.csdn.net/codename_cys/article/details/135318389?spm=1001.2014.3001.5502
- 大模型炼丹术：大模型微调总结及实现：https://zhuanlan.zhihu.com/p/673789772
- 大模型高效微调-LoRA原理详解和训练过程深入分析：https://zhuanlan.zhihu.com/p/702629428
- 百炼官方文档：https://bailian.console.aliyun.com/?tab=doc#/doc/?type=model&url=2844735

06 团队介绍

本文作者霄杨，来自淘天集团-用户场景营销技术团队。一支专注于探索AI等前沿技术与营销业务技术的融合，深度结合用户场景与营销业务的技术团队。依托大淘宝丰富的用户生态和多元化的消费场景，致力于通过技术创新提升用户体验，优化个性化营销能力，助力业务持续增长。通过AI驱动的精准推荐、场景化表达与动态策略调控，我们为用户创造更自然、更智能的购物旅程，为营销业务提供高效、敏捷的技术支撑，助力淘宝构建以用户为中心的全域营销技术体系。我们坚信技术是连接用户与价值的桥梁，持续探索创新边界，让营销更懂用户，用技术点亮每一个关键用户体验瞬间。

大淘宝技术
大淘宝技术官方账号
968篇原创内容
公众号

阅读 7130

留言

写留言

还没有留言，来写首评